Month 2 progress report

Baakhapaa: AI-powered pre-production intelligence system

Weeks 5 to 8 of 12 · Prepared for Baakhapaa · Reference: proposal §8

The four modules scheduled for Month 2 were finished ahead of schedule in Month 1 and reported then. This month went instead on the work the schedule had no room for: proper testing, the first run against live AI providers, and the first measurement of how well retrieval actually performs.

1. Month 2 deliverables

Wk	Build focus	Deliverable	Status
5	Storyboard generation	Shot types, camera notes, frame controls	Delivered Month 1
6	Collaboration and versions	Comment threads, roles, version diff	Delivered Month 1
7	Export system	PDF, Word, Final Draft `.fdx`, production package	Delivered Month 1
8	Subscription tiers	Free / Pro / Studio, three gateways, tier gating	Delivered Month 1
—	**Quality and measurement**	**Tests, retrieval evaluation, first live-key run**	**Done**

Each of the four was re-tested rather than rebuilt.

Week 5: Tests

Three parts of the backend had no tests, and they were three of the riskiest. The payment webhook grants a paid subscription and is the only endpoint that does so without a login. `updates.py` decides which fields a user may change. `rag.py` returns an empty result when it fails, so a fault there produces no error at all.

104 backend tests were written, and the 26 frontend components with no tests were covered. The suite now stands at **765 backend tests in 42 files** and **1,020 frontend tests in 54 files**.

Two security faults surfaced. In `versions.py` the permission check ran after the other error checks, so the difference between responses told a logged-in user whether two script versions existed and belonged to the same script, without access to either. And the command palette hid itself from logged-out visitors but still called the server; the resulting 401 logged them out.

Week 6: Running with real API keys

Two problems appeared the moment demo mode was switched off.

The test suite began calling the live API. Its configuration forces a mock database and skips payment providers, but nobody had done the same for the AI keys. A suite that finishes in seconds took 26 minutes, because the library retries each failure. The fix is less obvious than it looks: deleting the environment variable does not work, since the loader fills in whatever is missing, so it must be set to an invalid value instead.

The cost model was wrong by a factor of twelve. A storyboard image had been estimated at $0.040 assuming DALL-E 3; the model actually used costs $0.0033.

Item	Estimated	Measured
One storyboard frame	$0.040	**$0.0033**
24-frame storyboard	$0.96	**$0.08**
One script with a storyboard	$1.32	**~$0.44**

Images are roughly 18% of what a script costs to produce and text generation 82%, so shortening generated text saves far more than capping the number of frames.

Week 7: Measuring retrieval quality

Retrieval is the feature the product is built around, and nobody had measured it. A test set of 34 cases was assembled from the craft library and scored on precision@1 (whether the right answer came first), precision@3, and mean reciprocal rank.

The first run reported 82.4%, which was encouraging and wrong:

```
REAL QUERIES (what the editor actually sends, n=5)
  precision@1  20.0%     precision@3  60.0%

SANITY CHECK (an entry finds itself, n=29)
  precision@1  93.1%     <- should stay near 100%
```

Twenty-nine cases search using text that was itself used to build the index, so a correct answer is close to guaranteed. Only five resemble what the application really sends, and averaged together the easy majority hid the figure that mattered.

The groups are reported separately now and **the baseline is 20% precision@1**. It matches what the corpus loader had been showing all along: four built-in checks all reporting success, three of them ranking the wrong answer first.

Week 8: Streaming and character analysis

AI requests used to wait for the whole response before showing anything. Scene generation and rewriting now stream as the text arrives, with errors sent inside the stream, since once a response has started there is no status code left to report a failure with.

A view called Cast was added. Existing tools examine formatting, shape or scene order, and none help with a problem writers hit constantly: two characters who sound the same. Cast gathers each character's dialogue and reports line length, how much of their wording repeats, and how often they ask a question rather than make a statement.

Character	Lines	Words per line	Vocabulary	Asks
AARATI	30	4.1	0.65	0.17
KANCHHA	15	4.7	0.77	**0.40**
BABA	15	6.3	0.76	**0.00**

Kanchha asks a question in 40% of his lines; Baba never asks one. The difference shows as a number before it shows on the page. The view also displays the character description entered at project setup, which had only ever been used inside AI prompts.

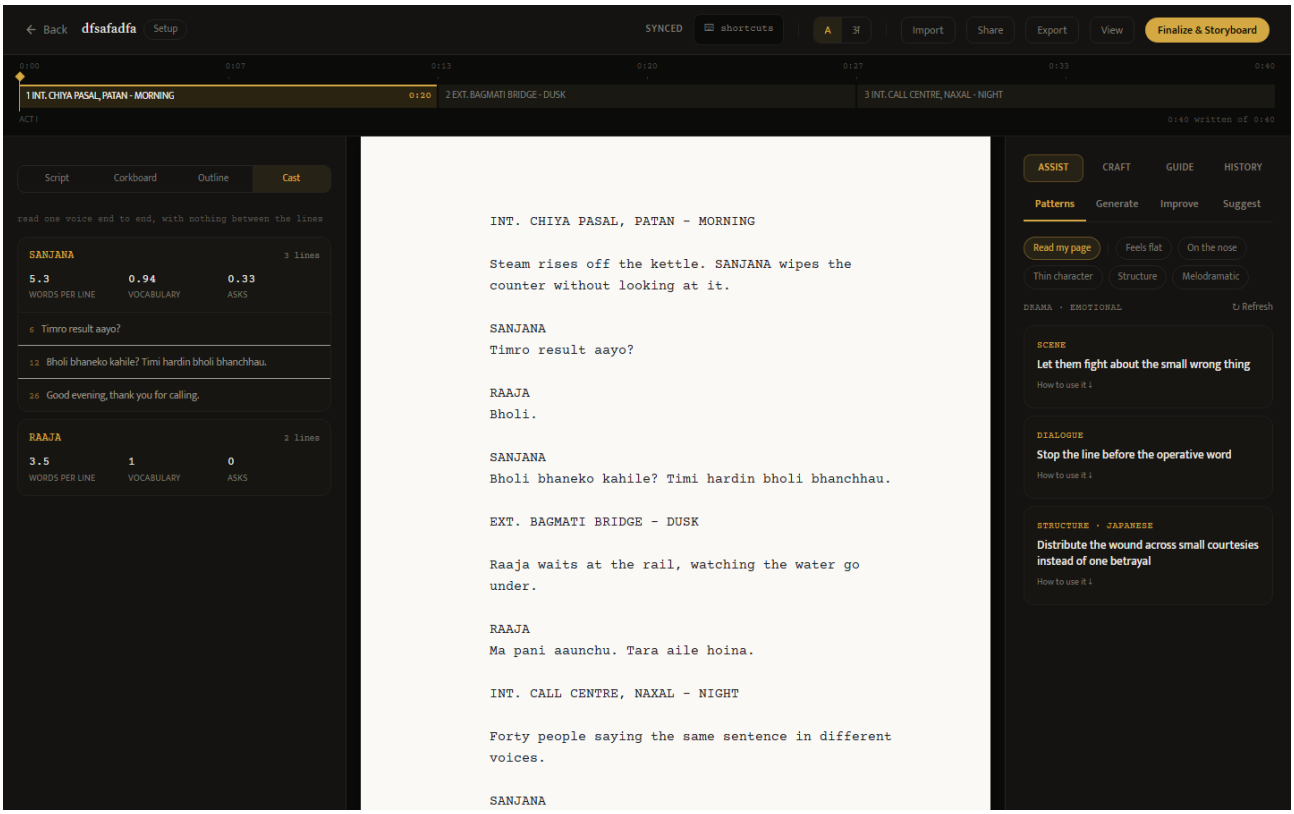

Figure 1. *Cast with one character opened. Each line carries its number in the script and clicking it moves the cursor there.*

2. Work beyond Month 2 scope

2.1 Faults found by writing a real screenplay

A complete short film was written inside the system and put through its own tools. Five faults came out that no test fixture had caught.

1. `FADE IN:` was counted as a scene, so a 16-scene script reported 17. The error carried into page statistics and the corpus figures shown to writers.
2. The craft panel analysed the wrong text: choosing a problem category made it examine the category description rather than the script, then report the result as found in the draft.
3. Renaming a scene had no effect on the scene list, so the timeline kept showing a heading no longer in the script.
4. Focus mode drew a 723-pixel page on a 1,274-pixel screen.
5. The editor URL was labelled as carrying a project identifier but carried a script one.

2.2 Interface changes

The four views moved into the left panel, so the script stays visible while reorganising. Page height now matches the pagination rule and screenplay margins were applied. Scene headings can be renamed on the timeline and act durations edited. Typewriter scrolling and custom cursors were added, and Word `.docx` import now sits alongside Final Draft, Fountain, text and PDF.

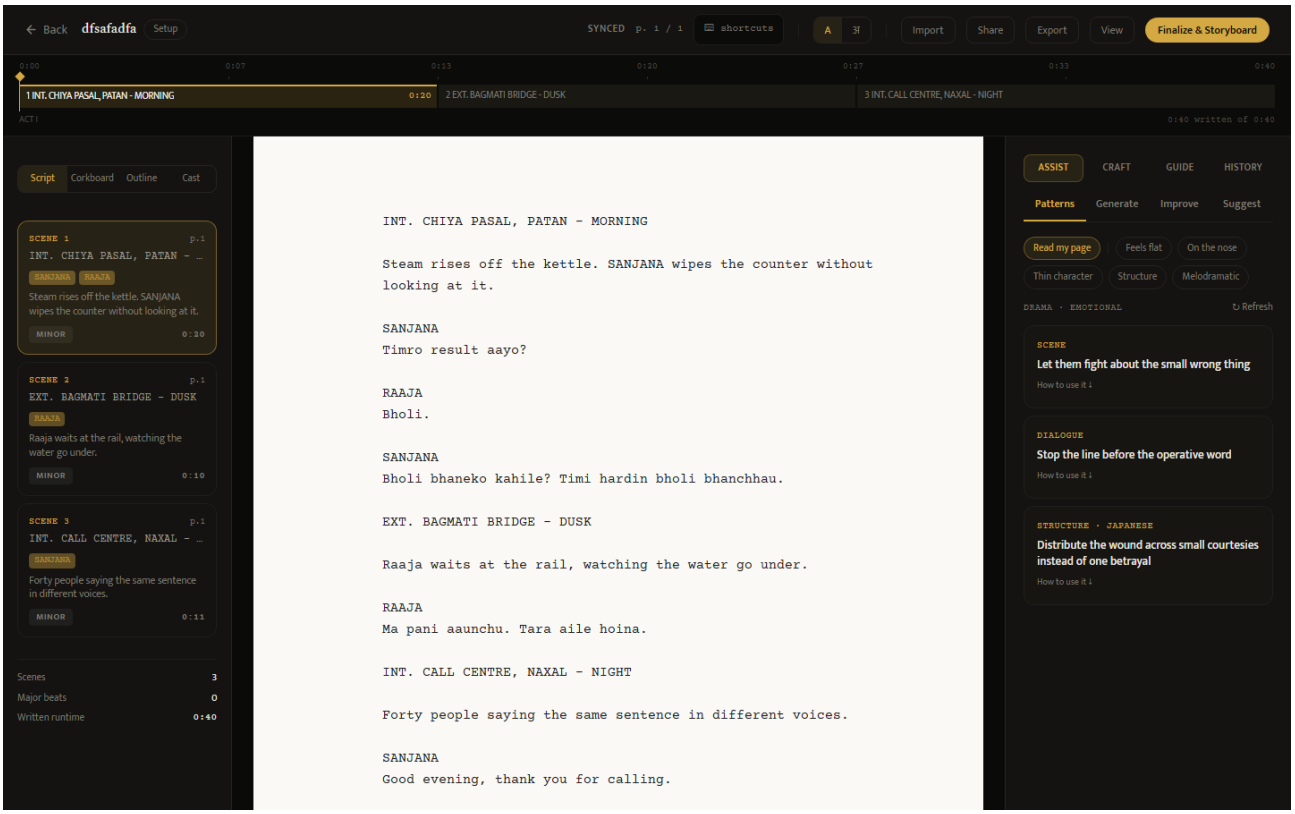

Figure 2. *The workspace at the end of Month 2, with the four views and scene cards on the left, the page in the middle and the assistant panel on the right.*

2.3 Documentation and decisions

Three open questions were settled: real-time collaborative editing was dropped from scope, leaving sharing, roles and comments; the encryption claim was rewritten to describe what the system does; and the tiers were fixed as free, pro and studio. The deployment guide, which described three migrations when there are four, was corrected.

3. Verification status

Both AI providers can be reached with live credentials and the image path has been run end to end at real cost. Text generation returns a billing error because the account has no credit, so streaming and the paid features remain verified against the mock provider only.

The system has not been deployed. It runs on a local database, which leaves four schema migrations unapplied, and no payment has been processed through any gateway. A cloud database, a live server and a first real payment are the work of Month 3, and every remaining estimate depends on them.

Figures were captured from the running system using `baakhapaa-frontend/scripts/capture-screenshots.mjs`.